Repository

Repository · Persist users behind collection-like API · Build with Waqar

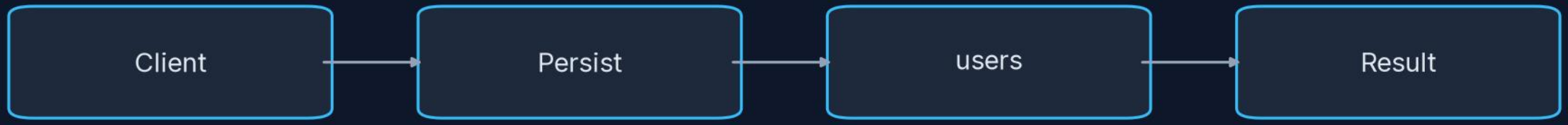
Today's Hook

- Problem: Persist users behind collection-like API
- Pattern: Repository — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Persist users behind collection-like API
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Repository: Persist users behind collection-like API

Real-World Scenario

- Running example: Persist users behind collection-like API
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.repository;

import com.buildwithwaqar.patterns.enterprise.repository.*;
/** Problem: persist users without leaking Map/SQL into services. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Repository problem: user persistence ===");
    var repo = new InMemoryUserRepository();
    repo.save(new User("1", "waqar@bwq.dev"));
    System.out.println(repo.findById("1").orElseThrow());
    System.out.println("missing=" + repo.findById("x"));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.repository;

import java.util.Collection;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

record User(String id, String email) {}

interface UserRepository {
    Optional<User> findById(String id);
    User save(User user);
    Collection<User> findAll();
}

/** Collection-like interface over persistence details. */
public final class InMemoryUserRepository implements UserRepository {
    private final Map<String, User> store = new ConcurrentHashMap<>();

    public Optional<User> findById(String id) { return Optional.ofNullable(store.get(id)); }
    public User save(User user) { store.put(user.id(), user); return user; }
    public Collection<User> findAll() { return store.values(); }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.enterprise.repository;

import com.buildwithwaqar.patterns.enterprise.repository.*;
/** Problem: persist users without leaking Map/SQL into services. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Repository problem: user persistence ===");
    var repo = new InMemoryUserRepository();
    repo.save(new User("1", "waqar@bwq.dev"));
    System.out.println(repo.findById("1").orElseThrow());
    System.out.println("missing=" + repo.findById("x"));
  }
}
```

Walkthrough Output

```
=== Repository problem: user persistence ===
User[id=1, email=waqar@bwq.dev]
missing=Optional.empty
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Repository.

Related Patterns

- Contrast Repository with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Repository → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.